

Maschinelles Lernen

Prof. Dr. Christine Preisach



Klassifikation

Klassifikation ordnet Beispielen (z.B. Zeilen aus einer Tabelle) diskreten Klassen zu

- Vorgegebene Klassen $Y = \{C_1, \dots, C_k\}$
- Gegeben Menge $X \times y \subset X \times Y$ bei der jedem Beispiel x_i die zugehörige Klasse zugeordnet ist:
 (x_i, y_i)
- **Fehlerfunktion** $f : (X \times Y) \times (X \rightarrow Y) \rightarrow \mathbb{R}$

Ziel:

Finde Modell

$$m : X \rightarrow Y,$$

das die **Fehlerfunktion** optimiert.

➔ Das Lernen eines Modells (Funktion) entspricht dem lösen eines **Optimierungsproblems!**

Klassifikation - Beispiel

Beispiel: Klassifikation von **Schwertlilien** (iris)

Klassen: $Y = \{setosa, versicolor, virginica\}$

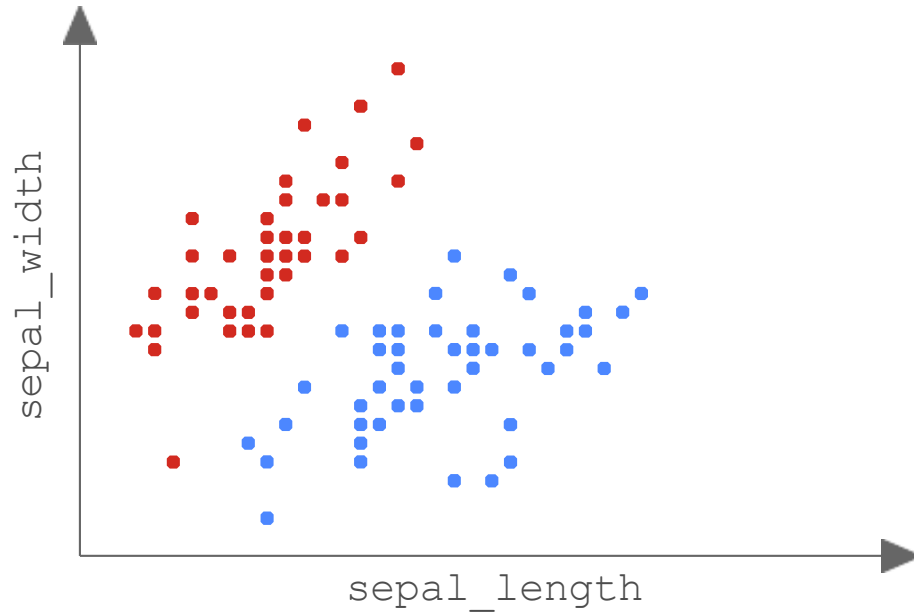
Menge $X \times y$ mit 150 Beispiele mit Spalte "species"

Fehlerfunktion: $f : \sum_{(x,y) \in X \times Y} \text{err}(y, f(x))$, $\text{err}(y, \hat{y}) = \begin{cases} 0, & \text{falls } y = \hat{y} \\ 1, & \text{sonst} \end{cases}$

f zählt die **Anzahl der Vorhersagefehler** des Modells m auf der Menge X (Trainingsmenge)

Ziel: Finde ein solches m mit **minimalem** f (Funktion=Modell das die wenigsten Fehler macht) →
Optimierungsproblem

Beispiel: Klassifikation von Schwertlilien



- Streudiagramm sepal_length und sepal_width
- Wir berücksichtigen hier nur Datenpunkte der Klasse setosa und versicolor
- **Ziel:** die beiden Klassen mit einer Funktion trennen sodass möglichst wenig Fehler (Fehlklassifikationen entstehen)

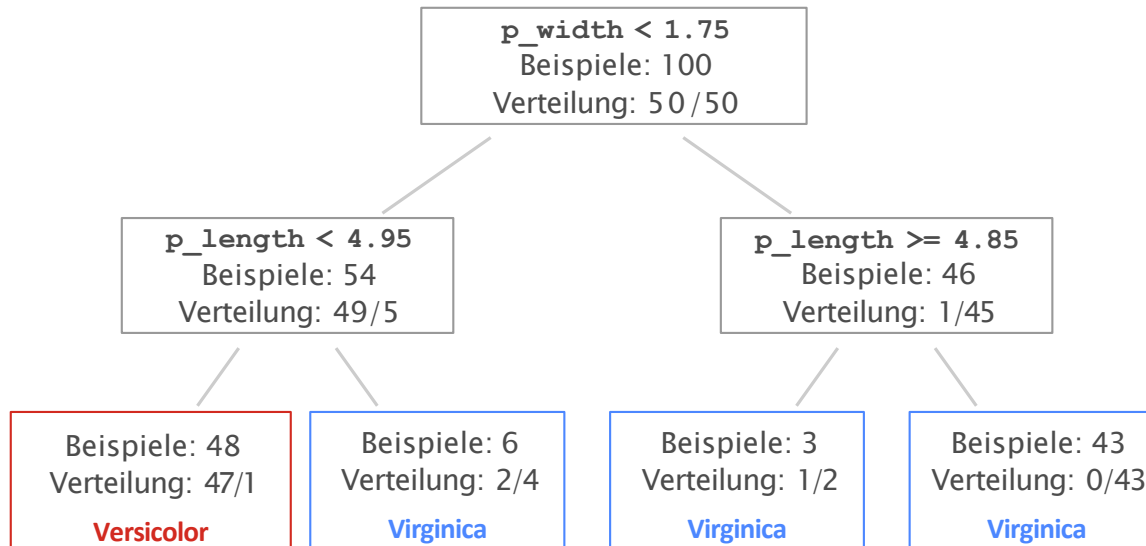
Entscheidungsbäume



Entscheidungsbäume

Entscheidungsbäume sind einfaches anschauliches Modell für Klassifikation:

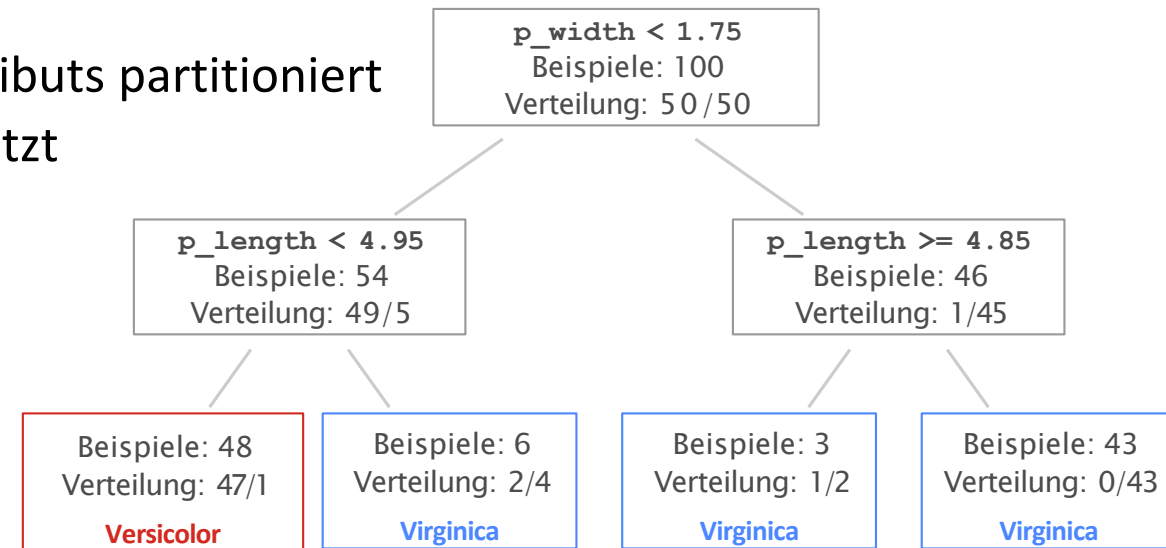
- Ganz oben ist die Wurzel des Baumes
- Die inneren Knoten und die Wurzel sind Entscheidungsknoten
- Die Blätter enthalten die Vorhersage für die Klasse
- Entscheidungsbäume können entweder von Experten manuell aufgeschrieben werden oder mit **Techniken des maschinellen Lernens automatisch gelernt** werden
- Ein Entscheidungsbaum enthält eine Menge von Regeln



Algorithmus - Entscheidungsbaum

Lernen eines Entscheidungsbaum:

- Anfangs gehören alle Trainingsdatensätze zur Wurzel
 - Ein Attribut wird ausgewählt (➔ siehe Splittkriterium)
 - Die Trainingsdaten werden unter Nutzung des Splittattributs partitioniert
 - Das Verfahren wird rekursiv für die Partitionen fortgesetzt bis ein Abbruchkriterium zutrifft
- ⇒ lokal optimierender Algorithmus



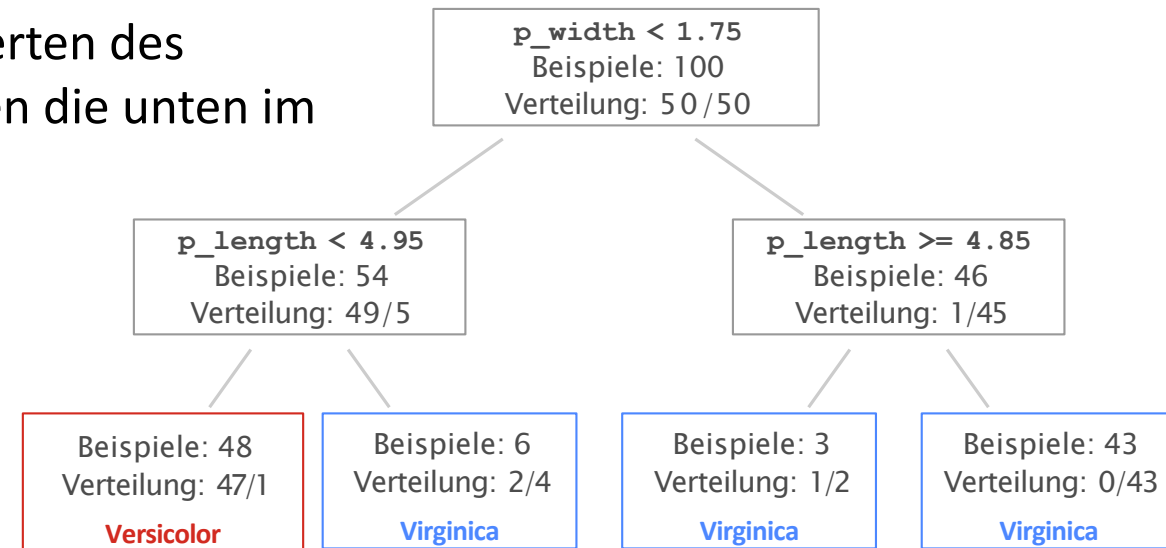
Beispiele für Abbruchbedingungen :

- keine weiteren Splittattribute
- Alle Trainingsdatensätze gehören zur gleichen Klasse (können nicht mehr weiter aufgeteilt werden)
- Bestimmte Baumtiefe erreicht

Algorithmus - Entscheidungsbaum

Vorhersage für ein unbekanntes Beispiel mit einem Entscheidungsbaum:

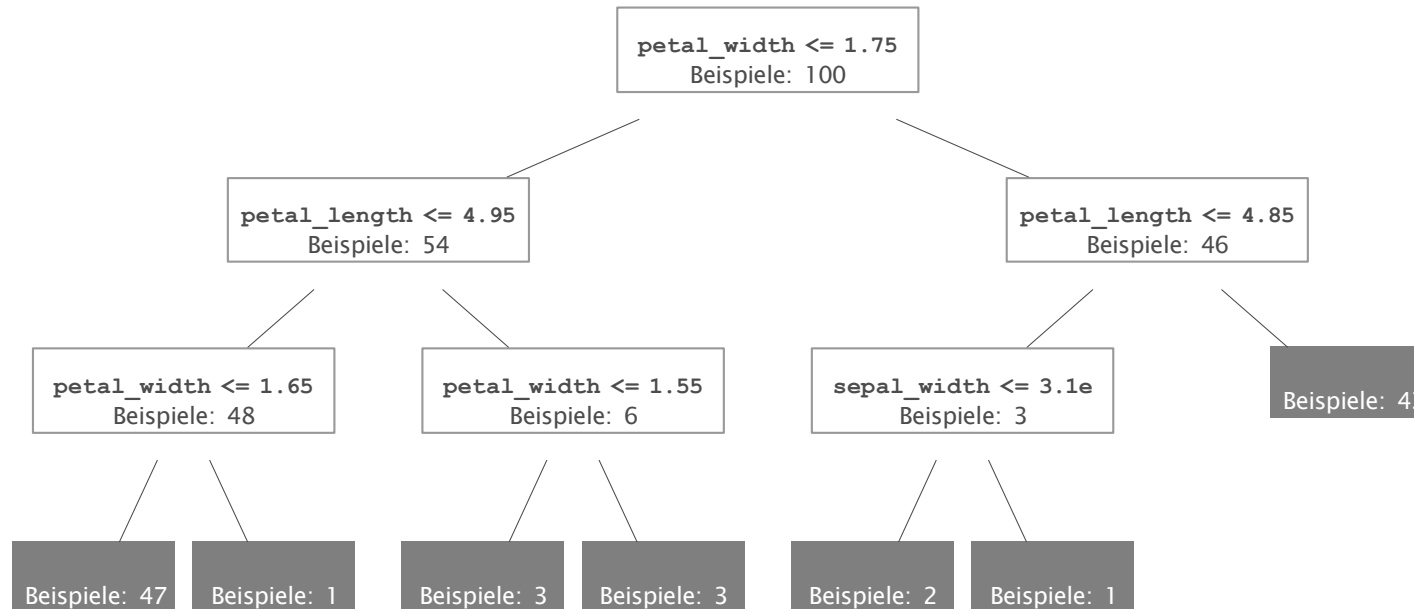
- Gelernter Entscheidungsbaum wird mit den Attributwerten des Beispiels durchlaufen und es wird die Klasse zugewiesen die unten im Blatt zu dem wir gelaufen sind, steht.
- Beispiel: petal_width = 1.2, petal_length = 3.2
➔ Zuweisung der Klasse Versicolor



Entscheidungsbäume

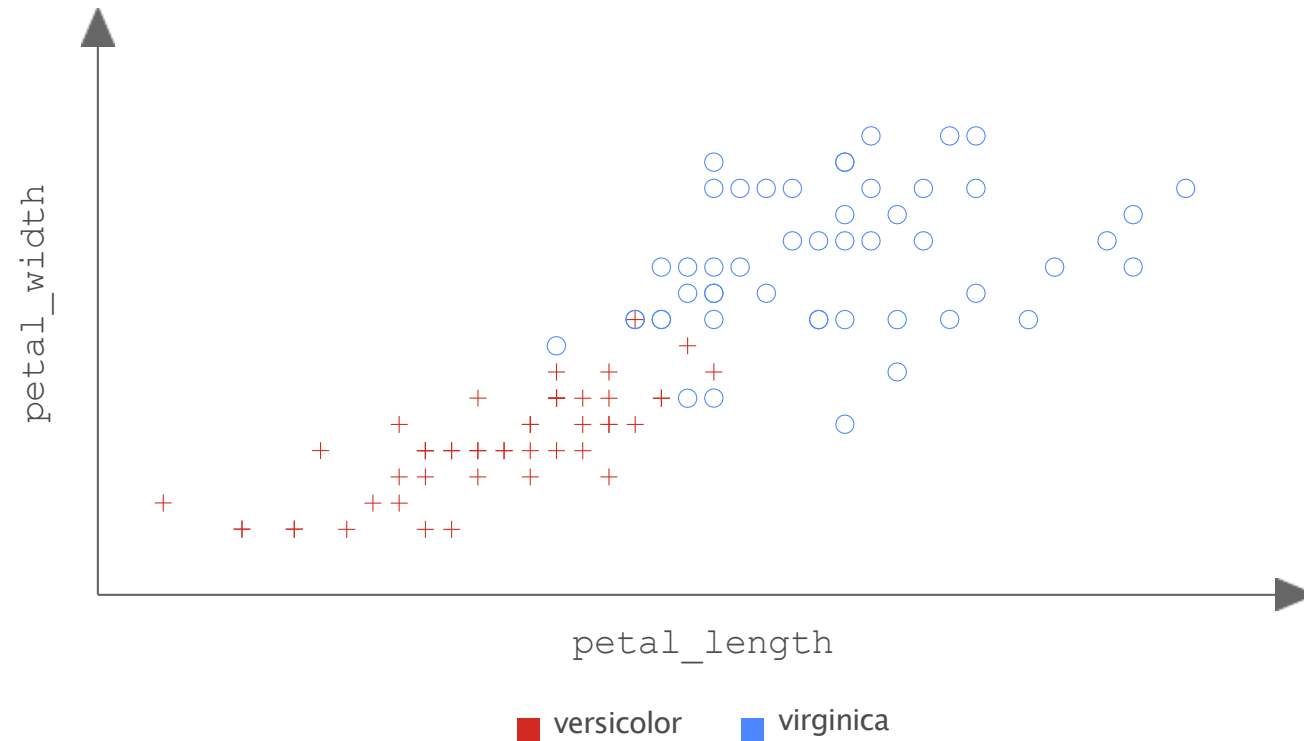
Ein größerer Baum mit 3 Inputvariablen

- In den Blättern ist zu sehen, dass dort teilweise nur sehr wenige Beispiele landen



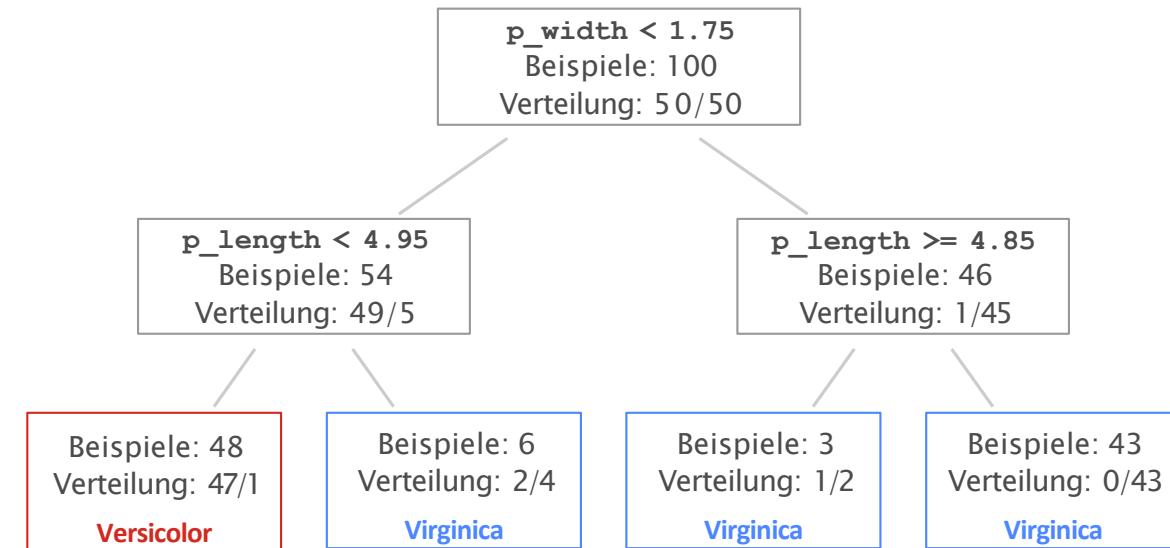
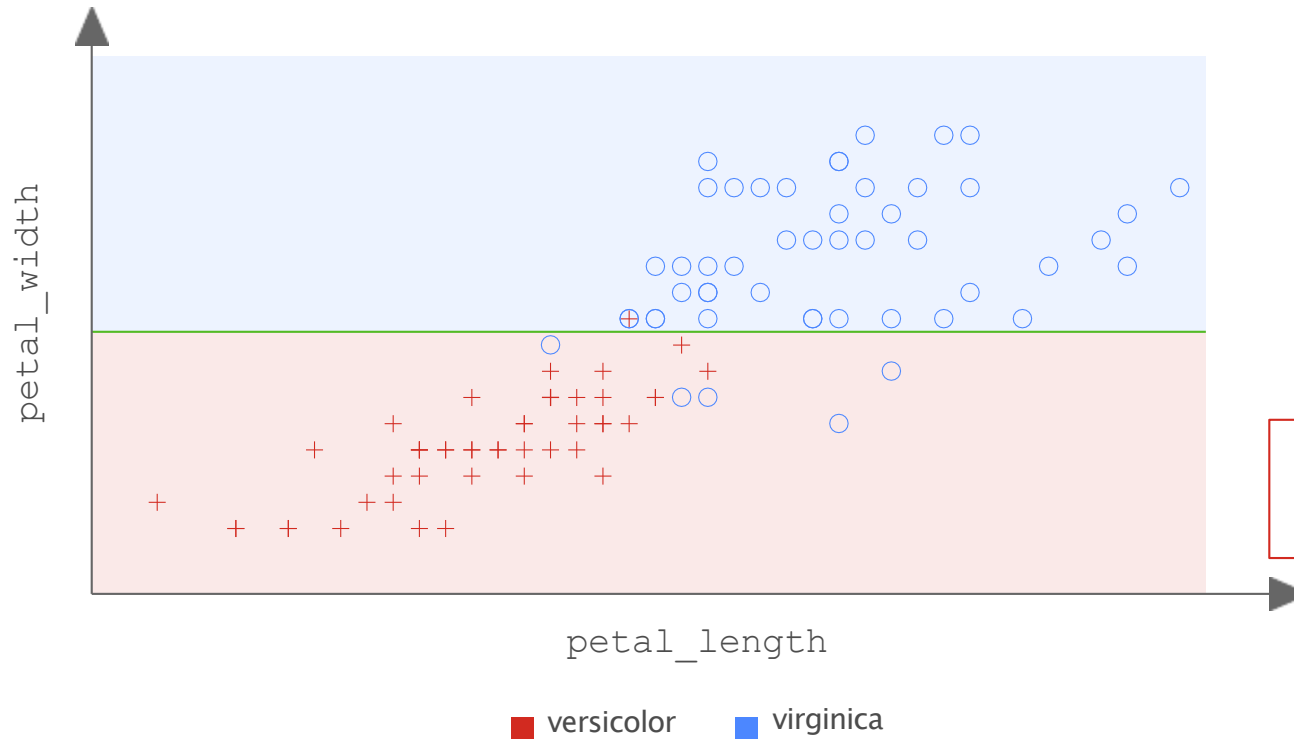
Entscheidungsbäume

Aufteilung der Daten durch eine Baum im Scatterplot



Entscheidungsbäume

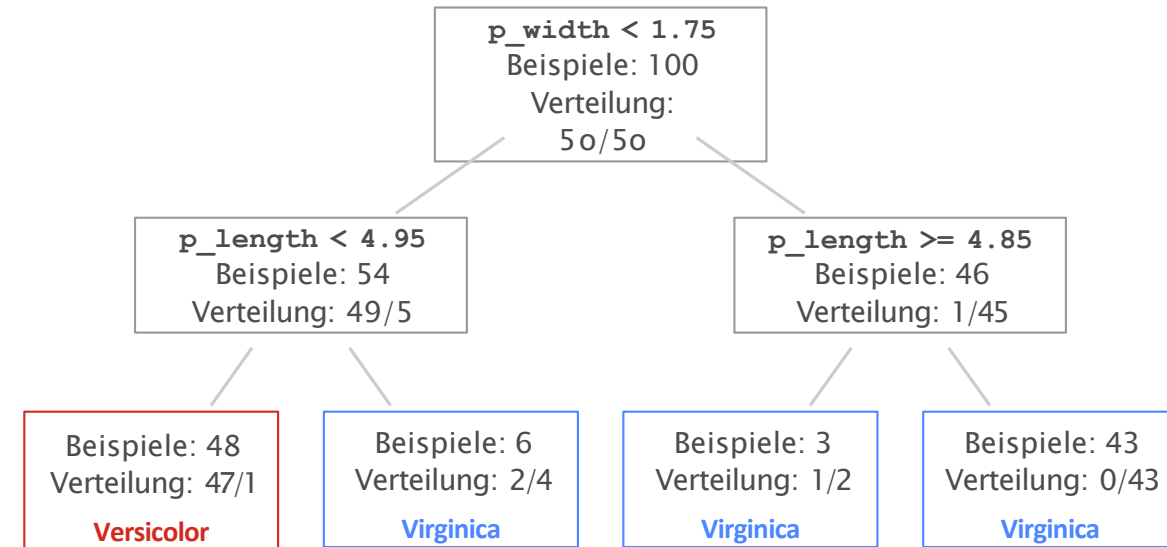
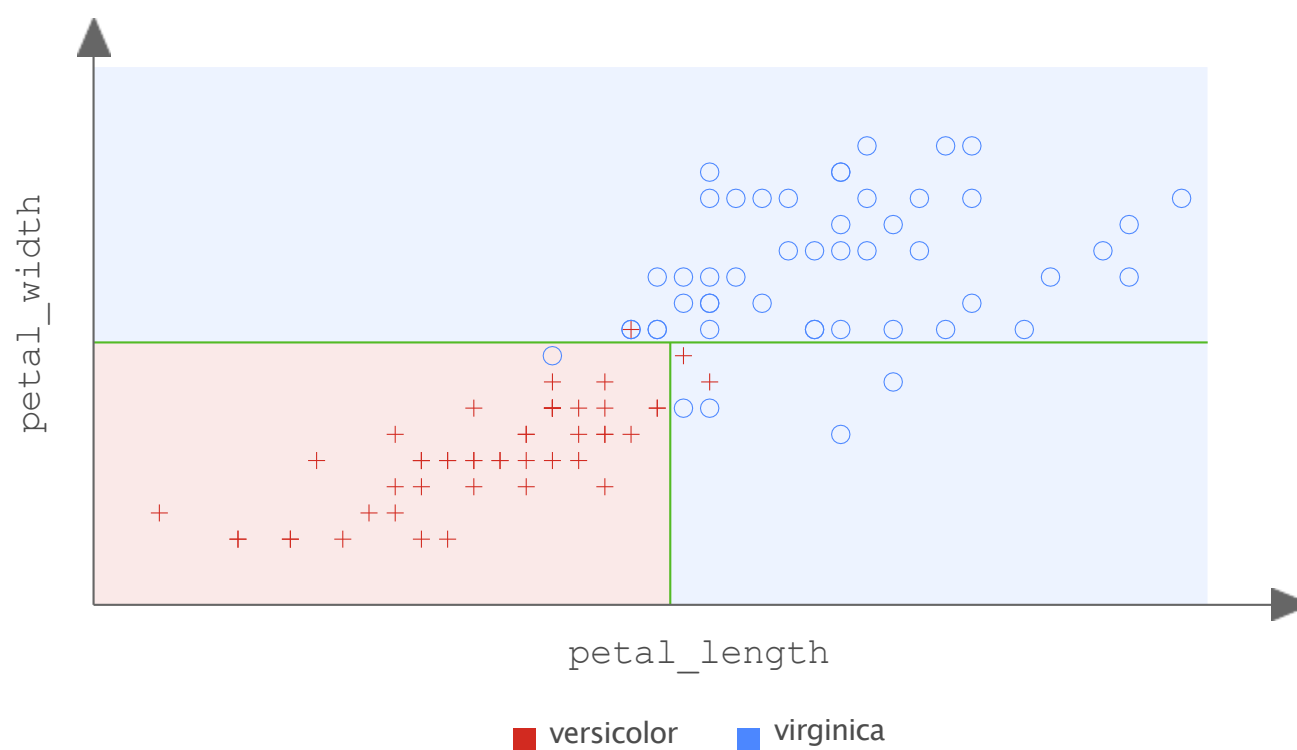
Aufteilung der Daten durch eine Baum im Scatterplot



Das Beispiel zeigt schrittweise die Aufteilung des Iris-Datensatzes über die Attribute `petal_width` und `petal_length`.

Entscheidungsbäume

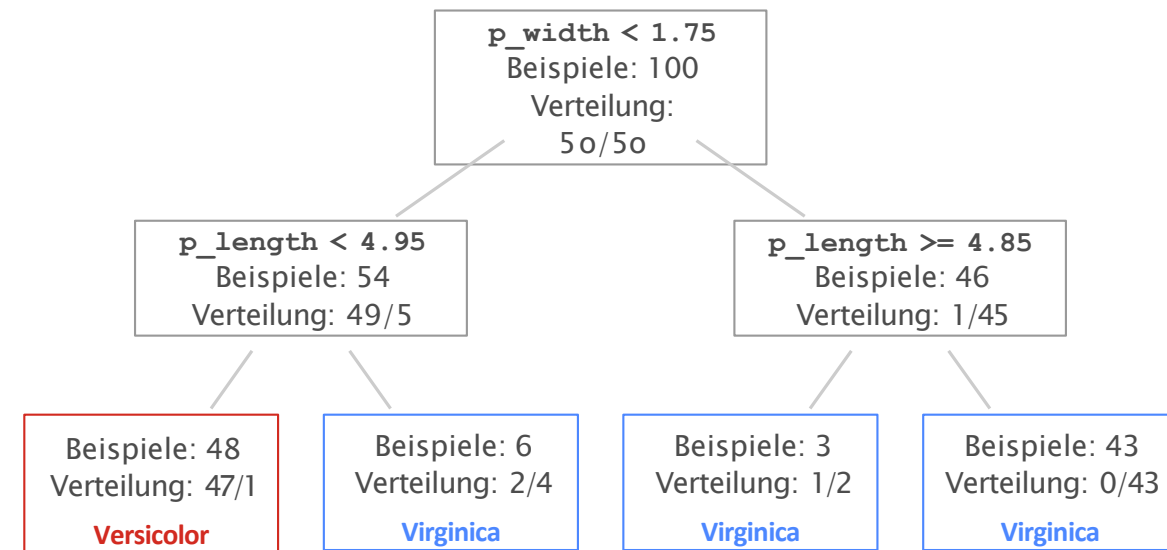
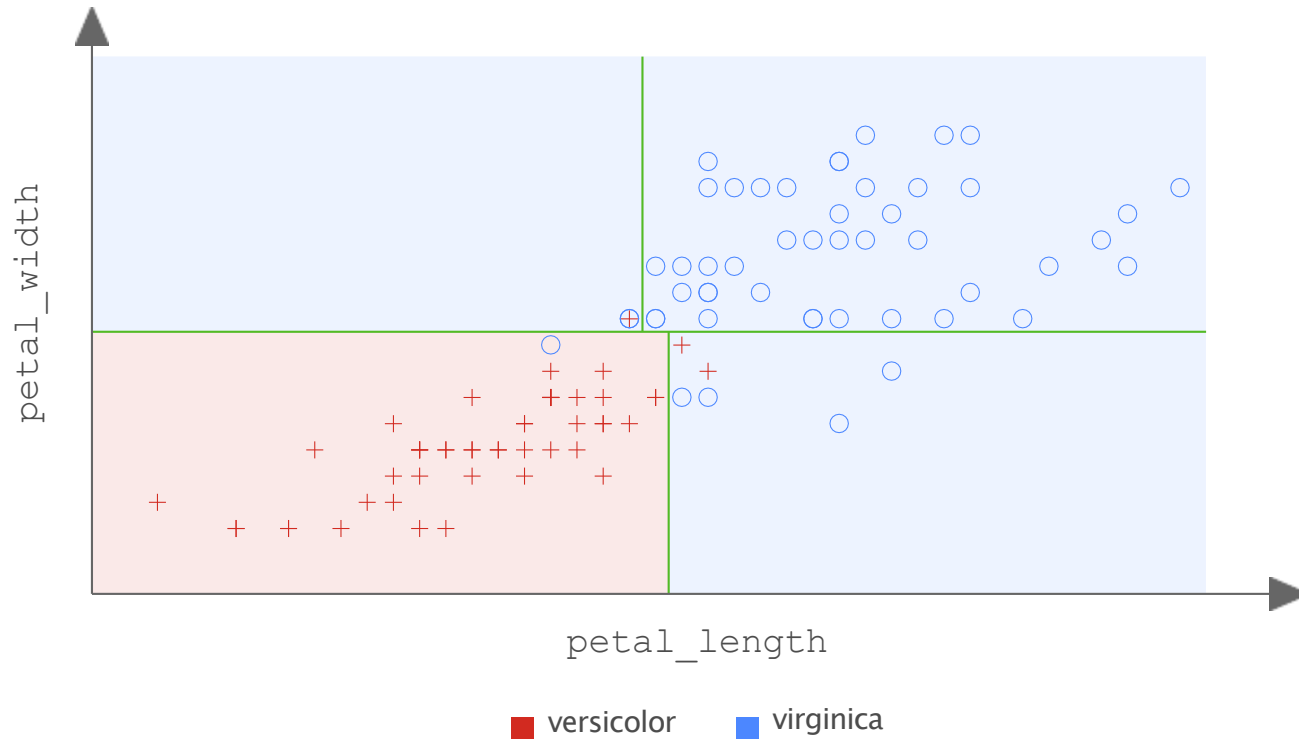
Aufteilung der Daten durch eine Baum im Scatterplot



Das Beispiel zeigt schrittweise die Aufteilung des Iris-Datensatzes über die Attribute `petal_width` und `petal_length`.

Entscheidungsbäume

Aufteilung der Daten durch einen Baum im Scatterplot



Das Beispiel zeigt schrittweise die Aufteilung des Iris-Datensatzes über die Attribute petal_width und petal_length.

Split-Kriterien

Frage: Wie finden wir eine gute Aufteilung der Daten?

- Gegeben sind die Trainingsdaten $\mathbf{X} = \mathbf{X}_{train} \times \mathbf{y}_{train}$
- Wir wollen einen Baum erstellen, der den **Trainingsfehler minimiert**
- Schrittweises Vorgehen (**rekursiv**):
 - Training eines Baumes:
 1. Finde bestes Attribut a und den Wert t zum Aufteilen von X
 2. Teile X in $X_a \leq t$ und $X_a > t$

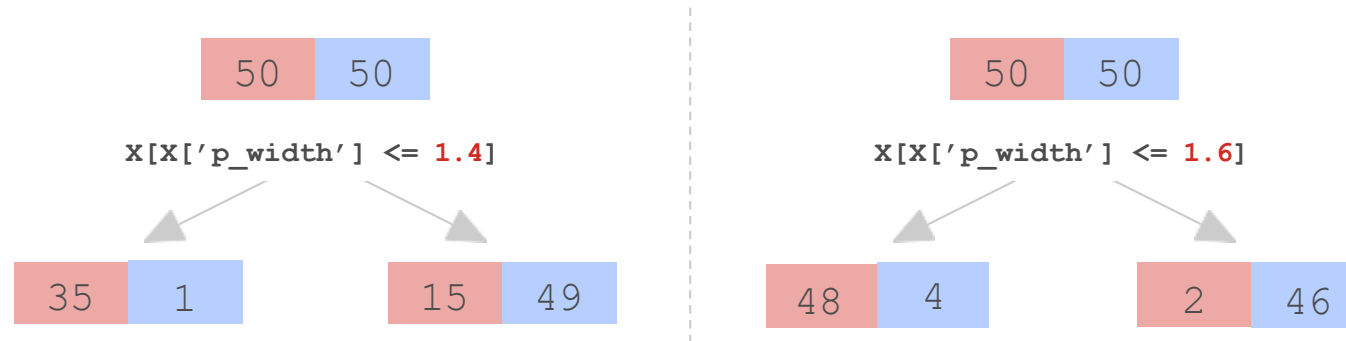
Frage: Wie finden wir das beste Attribut a und Wert t ?

- Wir probieren alle Attribute und deren Werte aus.

Split-Kriterien

Beispiel: Iris Daten mit Klassen virginica und versicolor

- Datensatz hat Klassenverhältnis 50:50
- Aufteilung nach Attribute p_width mit $t=1.4$ und $t=1.6$



Aber welche Aufteilung ist **besser**?

Split-Kriterien

Wir brauchen ein Maß um die Güte eines Splits zu bewerten:

Gini Index definiert eine Maßzahl für **Unreinheit**, diese möchten wir **minimieren**, d.h. wir möchten einen Knoten so aufteilen, dass möglichst **reine** Kind **Knoten** (bzgl. der Klassenzugehörigkeit) entstehen

$$G_A(S) = 1 - \sum_{i=1}^k P(C_i)^2, \quad P(C_i) = \frac{|X| \in C_i}{|X|}$$

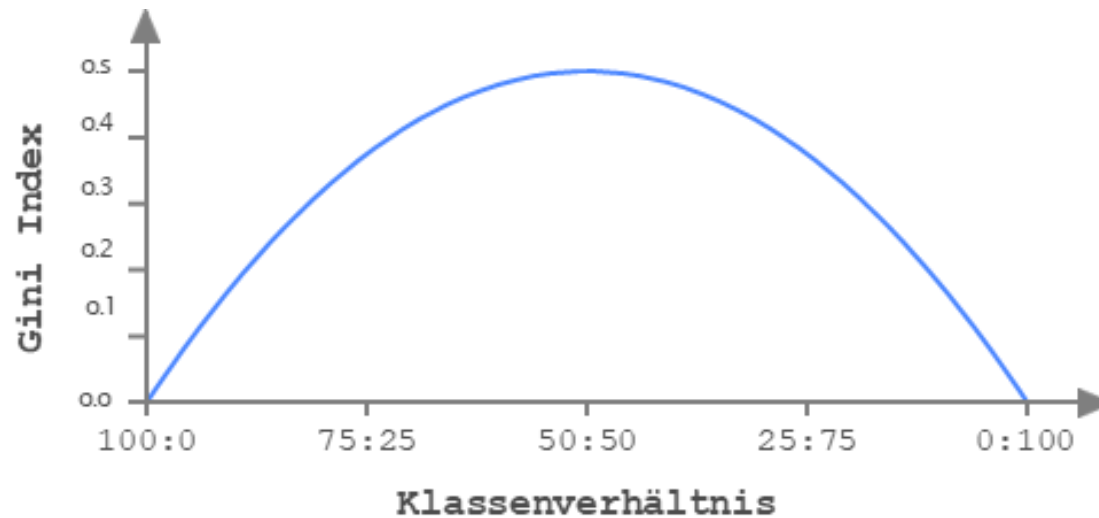
Mit **X** als Datenmenge in einem Knoten und mit den Klassen $C_1, \dots, C_k$, und $P(C_i)$ die Wahrscheinlichkeit für die Klasse C_i

Um den Gini Index für ein Attribut zu berechnen, müssen wir zuerst den **Gini Index für die die Aufteilung/Split (S_j) in linken und rechten Knoten berechnen** und dann die gewichtete Summe davon ermitteln. Wir gewichten mit dem Anteil an Beispielen, die im Split enthalten sind:

$$G_A(X) = \sum_{j=1}^m \frac{|S_j|}{|X|} \cdot G(S_j)$$

Split-Kriterien - Gini Index

- Kleiner Gini Index - geringe Unreinheit (Klassen können gut getrennt werden)
- Großer Gini Index - hohe Unreinheit (die Klassen können nicht gut getrennt werden)

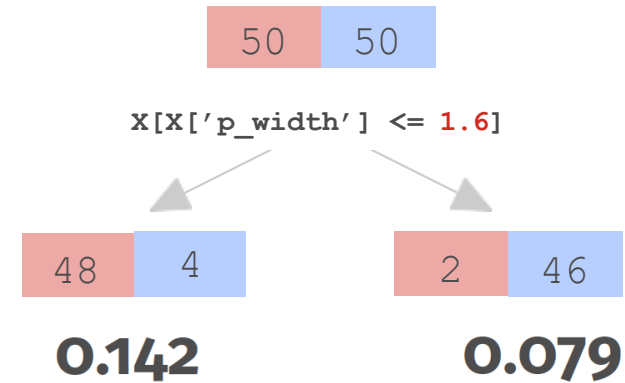
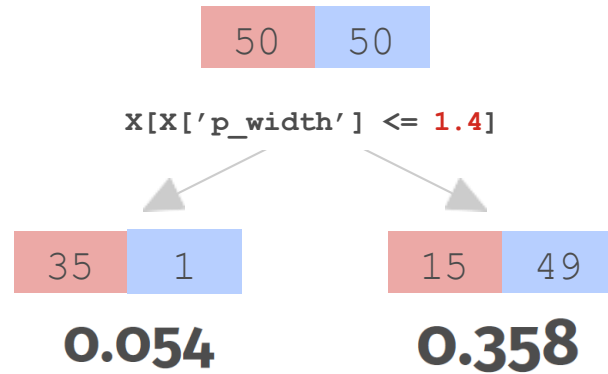


- Die Grafik zeigt den Verlauf des Gini Index (blau) für unterschiedliche Klassenverhältnisse. Eine reinere Aufteilung führt zu einem kleineren Gini Index Wert.
- Das Maximum hat der Gini Index beim Verhältnis 50:50.

Split-Kriterien

$$G_A(S) = 1 - \sum_{i=1}^{\kappa} P(C_i)^2,$$

$$P(C_i) = \frac{\text{Anzahl } X \in C_i}{|X|}$$



$$\text{gini}(35,1) = 1 - \left[\left(\frac{35}{36} \right)^2 + \left(\frac{1}{36} \right)^2 \right] = \mathbf{0.054}$$

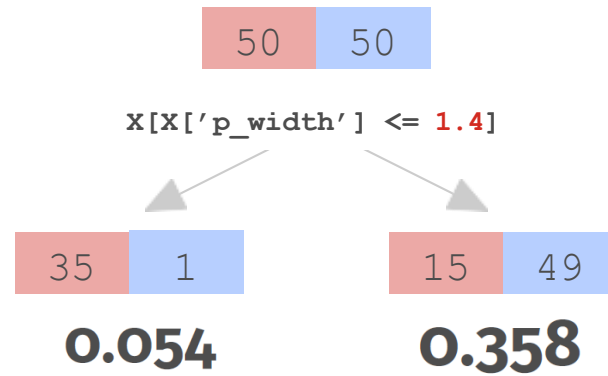
$$\text{gini}(15,49) = 1 - \left[\left(\frac{15}{64} \right)^2 + \left(\frac{49}{64} \right)^2 \right] = \mathbf{0.358}$$

$$\text{gini}(48,4) = \mathbf{0.142}$$

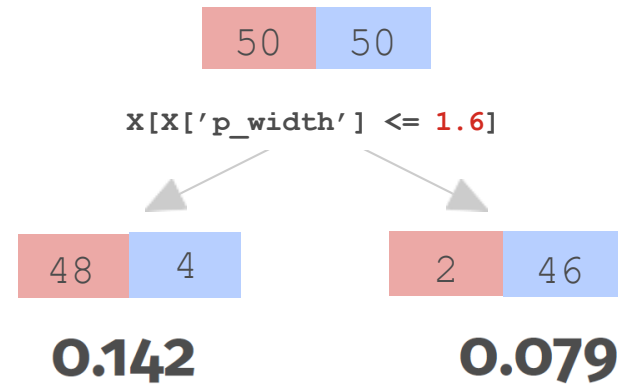
$$\text{gini}(2,46) = \mathbf{0.079}$$

Split-Kriterien

$$G_A(X) = \sum_{j=1}^m \frac{|S_j|}{|X|} \cdot G(S_j)$$



$$\frac{36}{100} \cdot 0.054 + \frac{64}{100} \cdot 0.358$$
$$= \mathbf{0.248}$$



$$\frac{52}{100} \cdot 0.142 + \frac{48}{100} \cdot 0.079$$
$$= \mathbf{0.112}$$

➔ Aufteilung mit `petal_width <= 1.6` liefert besseres Ergebnis als mit 1.4

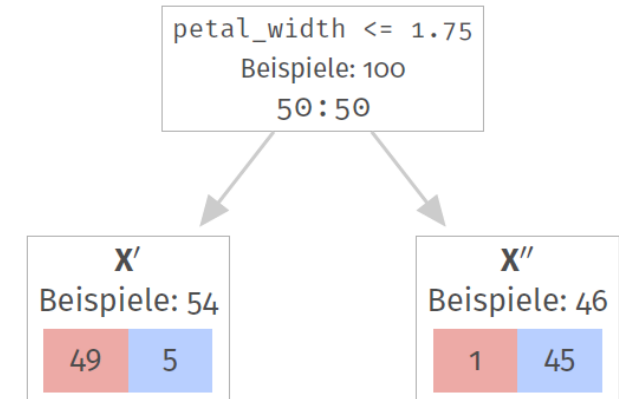
Split-Kriterien

	Bedingung: $\leq t$			Bedingung: $> t$			Σ
	#Versi	#Virgi	gini	#Versi	#Virgi	gini	
petal_width $\leq$ 1.0	7	0	0.000	43	50	0.497	0.462
petal_width $\leq$ 1.1	10	0	0.000	40	50	0.494	0.444
petal_width $\leq$ 1.2	15	0	0.000	35	50	0.484	0.412
petal_width $\leq$ 1.3	28	0	0.000	22	50	0.424	0.306
petal_width $\leq$ 1.4	35	1	0.054	15	49	0.359	0.249
petal_width $\leq$ 1.5	45	3	0.117	5	47	0.174	0.147
petal_width $\leq$ 1.6	48	4	0.142	2	46	0.080	0.112
petal_width $\leq$ 1.7	49	5	0.168	1	45	0.043	0.110
petal_width $\leq$ 1.8	50	16	0.367	0	34	0.000	0.242
petal_width $\leq$ 1.9	50	21	0.417	0	29	0.000	0.296
petal_width $\leq$ 2.0	50	27	0.455	0	23	0.000	0.351
petal_width $\leq$ 2.1	50	33	0.479	0	17	0.000	0.398
petal_width $\leq$ 2.2	50	36	0.487	0	14	0.000	0.419
petal_width $\leq$ 2.3	50	44	0.498	0	6	0.000	0.468
petal_width $\leq$ 2.4	50	47	0.500	0	3	0.000	0.485
petal_width $\leq$ 2.5	50	50	0.500	0	0	0.000	0.500

➔ Minimaler Gini Index Wert für petal_width $\leq$ 1.7

Split-Kriterien

- Es gibt noch andere Splitkriterien z.B. :
- **Entropie** (Informationsgehalt) und
- **Information Gain** (Informationsgewinn, den wir erhalten wenn wir das Attribut a mit dem Wert t zum Teilen benutzen)



Zusammenfassung

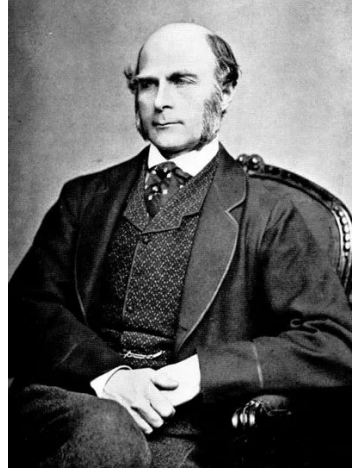
- Ein Entscheidungsbaum besteht aus einer Wurzel, mehreren inneren (Kind) Knoten und Blättern.
- Die inneren Knoten enthalten Entscheidungen bzgl. einer Variable die zur weiteren Aufteilung der Daten in den nächsten Knoten führen. Man nutzt Maße wie z.B. Gini Index um die Güte jedes Splits zu bewerten.
- Die Blätter enthalten die Klassifikation in bestimmte Klassen.
- Entscheidungsbäume werden aus den Trainingsdaten automatisiert gelernt, Sie enthalten Regeln, die zur Klassifikation von Daten genutzt werden können.

Vorteile und Nachteile

- Entscheidungsbäume lassen sich **einfach erklären**. Manche sind der Ansicht, dass Entscheidungsbäume die menschliche Entscheidungsfindung besser widerspiegeln als andere Ansätze.
- Entscheidungsbäume lassen sich **grafisch darstellen** und sind selbst für Laien leicht zu interpretieren (vor allem, wenn sie klein sind).
- Leider erreichen Bäume im Allgemeinen nicht die gleiche Vorhersagegenauigkeit wie einige der anderen Regressions- und Klassifizierungsansätze.
- Durch die Aggregation vieler Entscheidungsbäume lässt sich die Vorhersageleistung von Bäumen jedoch erheblich verbessern => Ensemble-Lernen mit Bäumen (z. B. Bagging, Random Forests und Boosting).
- Diese Methoden lernen mehrere Bäume, die dann kombiniert werden, um eine einzige Konsensvorhersage zu liefern.
- Die Kombination von vielen Bäumen kann oft zu große Verbesserungen der Vorhersagegenauigkeit führen, allerdings auf Kosten der Interpretierbarkeit.

Schwarm Intelligenz?

- Francis Galton hatte einige Daten von der „West of England Fat Stock and Poultry Exhibition“ im Jahr 1906 gesammelt.
 - 787 Besucher dieser Ausstellung schätzten das Gewicht eines Ochsen.
 - Ihre durchschnittliche Schätzung lag bei 1.197 Pfund.
 - Das tatsächliche Gewicht des Ochsen: 1.198 Pfund.
-
- **Offene Fragen:**
 - Sind Gruppen besser im Schätzen als Einzelpersonen?
 - Wenn ja, unter welchen Bedingungen?



Schwarmintelligenz?

- Meistens können Menschenmengen genauere Vorhersagen treffen, aber manchmal liegen sie auch völlig daneben.
 - Wir wollen dies verstehen, indem wir herausfinden, wann die Masse genaue Vorhersagen treffen kann.
-
- Der Schlüssel dazu ist **die Vielfalt der Menschen/Schätzungen**.
 - **Unabhängigkeit der Urteile/Schätzungen**.
 - Fehler heben sich gegenseitig auf, d. h., individuelle Schwächen werden eliminiert.



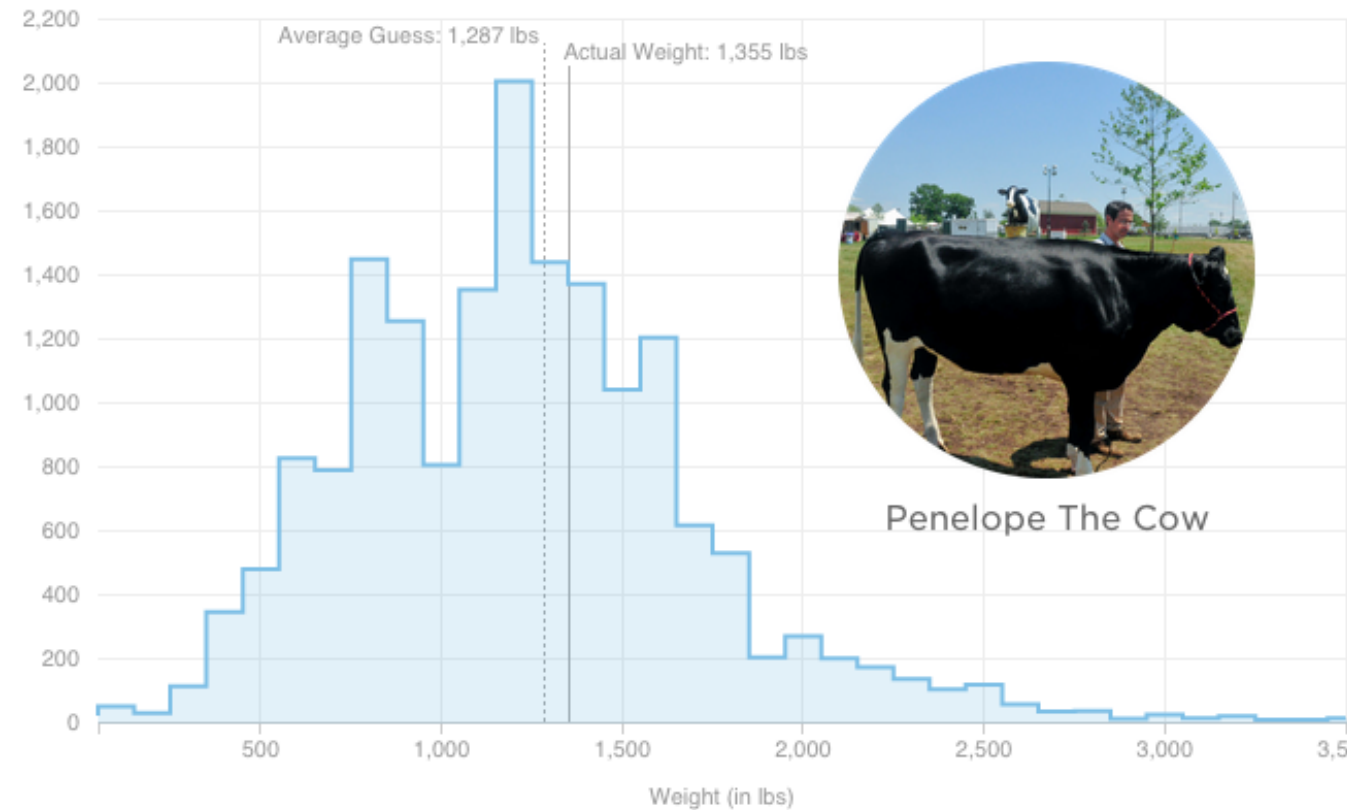
Schwarmintelligenz

- Im Jahr 2015 wurde das Experiment wiederholt:
- Über 17.000 Menschen nahmen daran teil
- Durchschnittliche Schätzung 1.287
- Tatsächliches Gewicht 1.355

How Much Does This Cow Weigh?

(All People)

Number Of Guesses



Penelope The Cow

Quelle: [estimatingWeightOfACow](#)

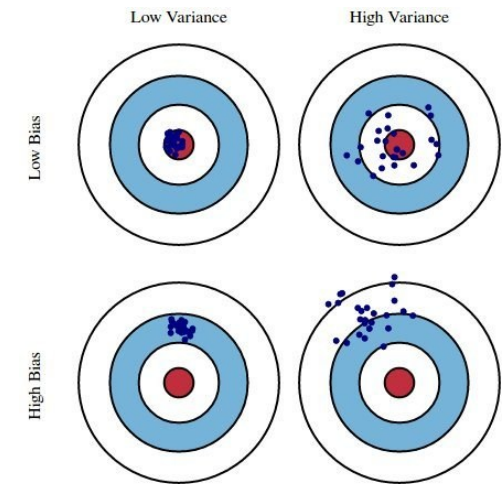
Verschiedene Modelle

Performance

- In der Regel ist keiner der Klassifikatoren perfekt, entweder weil sie einen **hohen Bias-Fehler** aufweisen oder weil sie **einen großen Varianzfehler** haben.
- **Aber:** Beispiele, die von einem Klassifikator nicht korrekt klassifiziert werden, können von den anderen Klassifikatoren korrekt klassifiziert werden.

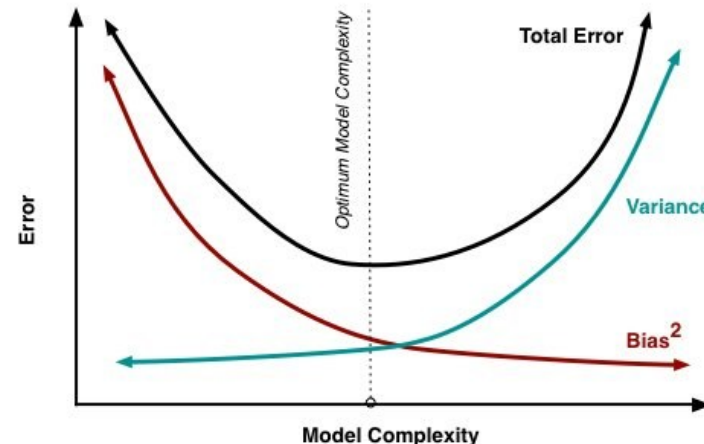
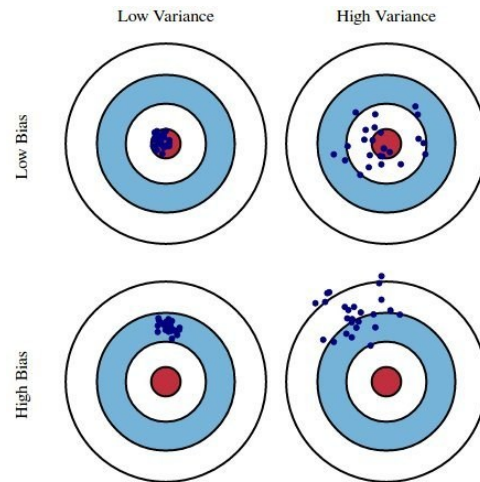
Idee des Ensemble-Lernens:

- Versuchen den Bias und/oder Varianz von Klassifikatoren (schwachen Lernern) zu reduzieren, indem **mehrere Modelle kombiniert werden**, um einen **starken Lerner** (oder **ein Ensemble-Modell**) zu lernen, der eine bessere Leistung erzielt.
- **Wir kombinieren verschiedenen** Klassifikatoren, um das endgültige Ergebnis zu erhalten.



Varianz – Bias Tradeoff

- Modelle können aus folgenden Gründen Fehler erzeugen:
 - Der **Varianzfehler** ist ein Fehler, der durch **die Empfindlichkeit gegenüber kleinen Schwankungen** im Trainingssatz entsteht. Eine hohe Varianz kann dadurch entstehen, dass ein Algorithmus das zufällige Rauschen in den Trainingsdaten modelliert (führt zu Overfitting).
 - Der **Bias-Fehler** ist ein Fehler, der durch **falsche Annahmen** im Lernalgorithmus entsteht. Ein hoher Bias kann dazu führen, dass ein Algorithmus **die relevanten Zusammenhänge** zwischen Merkmalen und Zielausgaben **übersieht** (Underfitting). Z. B. falsche Annahme einer linearen Beziehung
- Der **Bias-Varianz-Tradeoff** ist ein zentrales Problem beim überwachten Lernen
- Das Ziel ist es, Modelle zu finden, die sowohl **die Muster** in den Trainingsdaten **genau erfassen** als auch **bisher unbekannte Daten gut vorhersagen** .

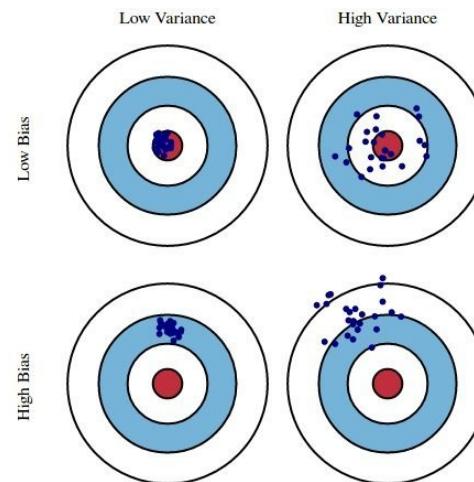


<https://www.kdnuggets.com/2016/08/bias-variance-tradeoff-overview.html>

Varianz und Bias

Wie lassen sich Streuung und Bias erkennen:

- **Visuelle Überprüfung:** vorhergesagte Werte den tatsächlichen Werten gegenüberstellen.
 - **Starker Bias:** Vorhersagen weichen durchweg stark von den tatsächlichen Werten ab.
 - **Hohe Varianz:** Vorhersagen sind weit um die tatsächlichen Werten entfernt.
- **Kreuzvalidierung:** Nutzung von k-fold crossvalidation, um Bias und Varianz zu schätzen.
 - **Hoher Bias:** Durchweg schlechte Leistung über alle Folds hinweg.
 - **Hohe Varianz:** Große Schwankungen in der Leistung über die Folds hinweg.



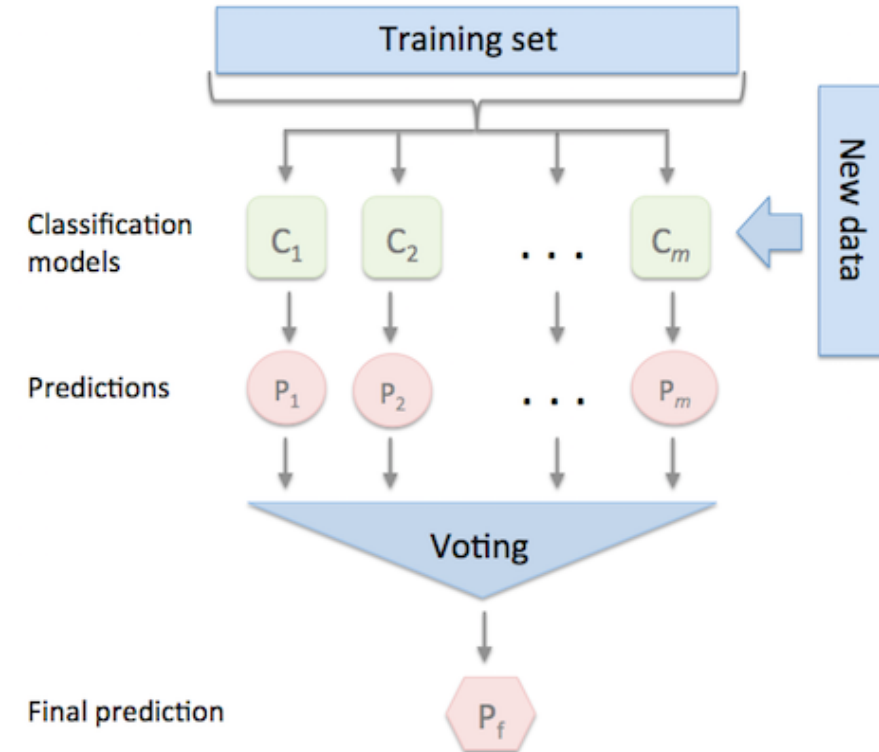
<https://www.kdnuggets.com/2016/08/bias-variance-tradeoff-overview.html>

<https://towardsdatascience.com/bias-variance-dilemma-74e5f1f52b12>

Einfaches Ensemble Modell: Voting

Einfache Ensemble-Verfahren: Kombination der Vorhersagen mehrerer Modelle.

- Geeignet für Klassifizierung und Regression: Bei einem Voting-Ensemble wird eine Vorhersage getroffen, die dem Durchschnitt mehrerer anderer Modelle entspricht.
- Bei der Klassifizierung werden bei einem **Hard-Voting-Ensemble** die Anzahl der Stimmen für jede **Klasse** aller Modelle addiert und die Klasse mit den meisten Stimmen (Stimmenmehrheit) vorhergesagt.
- Bei einem **Soft-Voting-Ensemble** werden die durchschnittlichen vorhergesagten **Wahrscheinlichkeiten** für jede Klasse berechnet und die Klasse mit der höheren Wahrscheinlichkeit vorhergesagt.



Einfaches Ensemble Modell: Voting

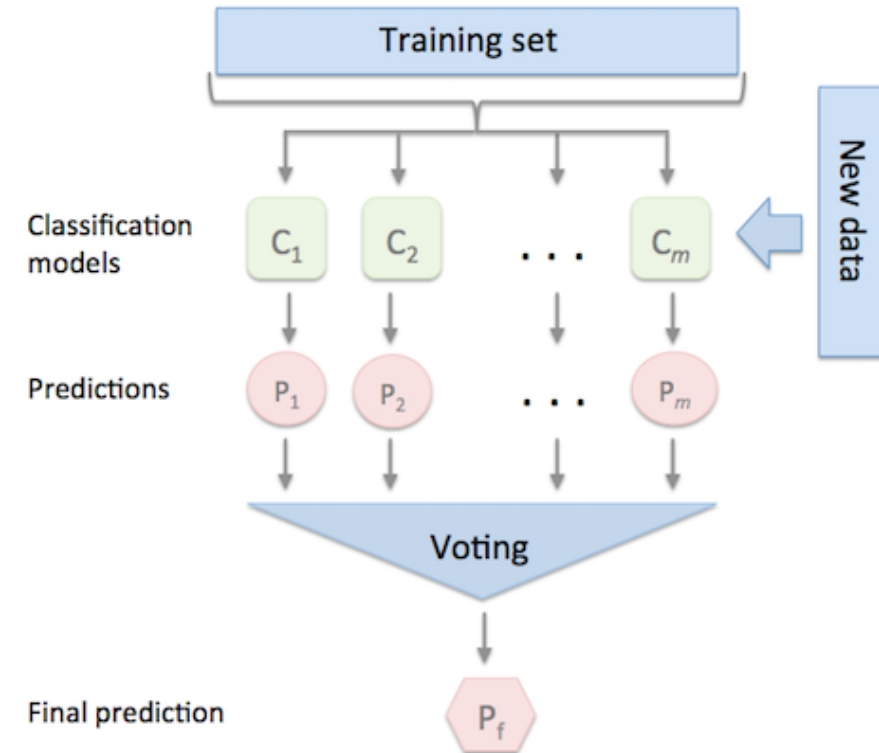
Vorgehensweise:

- mehrere Modelle auf einem Trainingsatz (mit unterschiedlichen Algorithmen) trainieren.
- Wende jedes Modell auf einen Testdatensatz an.
- Verwende Voting für die Vorhersagen, d. h. Mehrheitsentscheidung der Klassen oder die höhere durchschnittliche Wahrscheinlichkeit für Klassen

Empfehlung – nutzen wenn:

- alle Modelle im Ensemble **im Allgemeinen eine gute Leistung** aufweisen.
- Dies führt zu einer höheren Genauigkeit als bei jedem einzelnen im Ensemble verwendeten Modell.
- **Es ist nicht garantiert, dass das Voting-Ensemble eine bessere Leistung erbringt als jedes einzelne Modell.**

Varianten: gewichtetes Voting



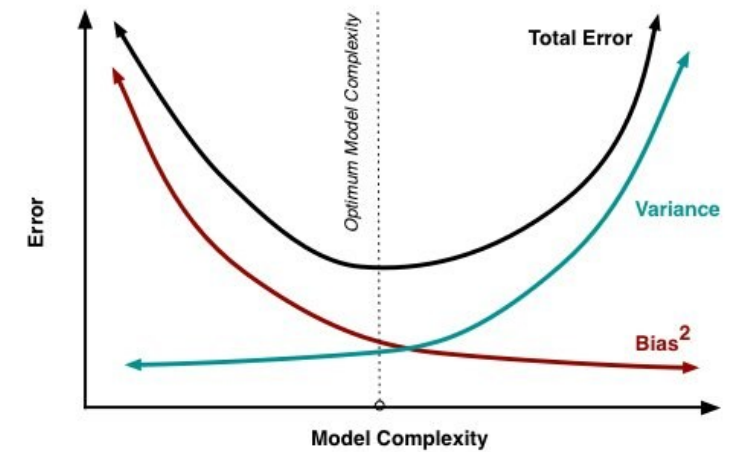
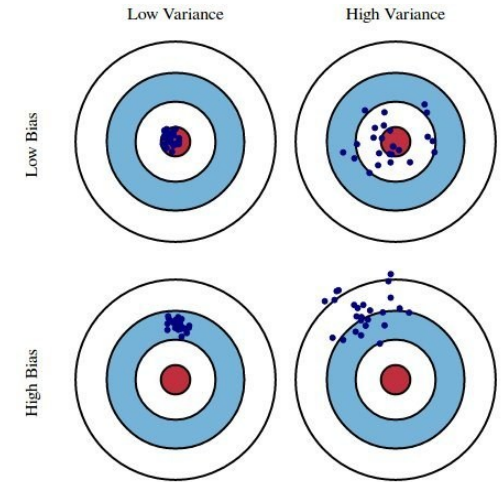
Andere Ensemble Learning Verfahren

Ensemble-Verfahren, die die Varianz minimieren und Overfitting vermeiden:

- Voting
- Bagging
- Random Forests

Ensemble-Methoden, die den Bias (und die Varianz) minimieren:

- Boosting (XGBoost)



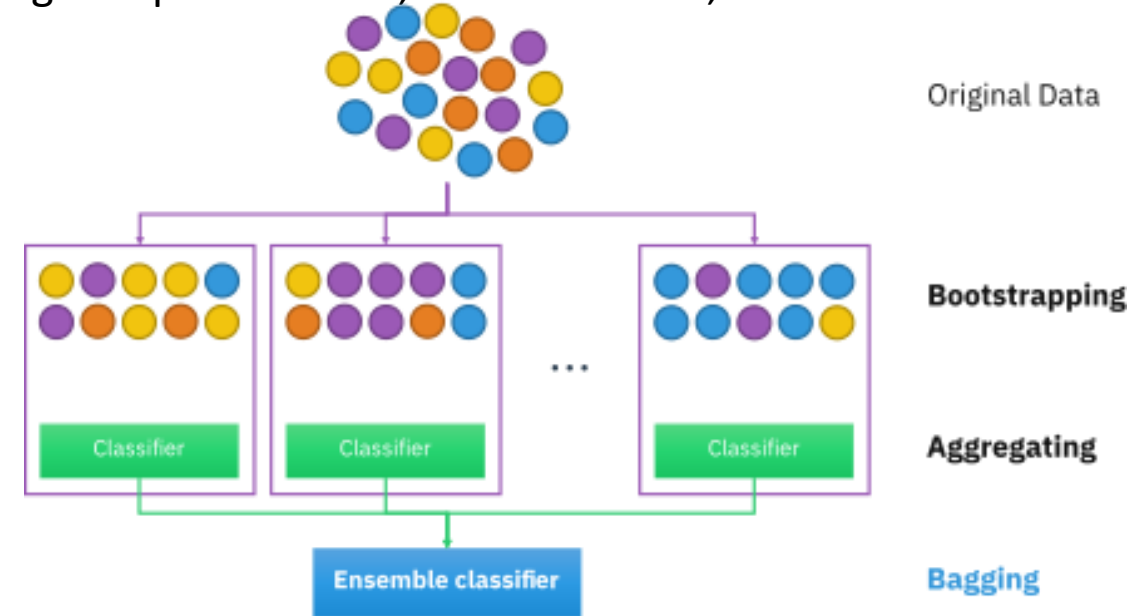
<https://www.kdnuggets.com/2016/08/bias-variance-tradeoff-overview.html>

Bagging Verfahren

- Ein Ensemble-Lernalgorithmus (Meta-Lerner) für Klassifikation und Regression, der entwickelt wurde, um die Vorhersagegenauigkeit und -stabilität zu verbessern, **die Varianz zu reduzieren und Overfitting zu vermeiden**, erfunden von Leo Breiman (1994)
- Verwendet nicht immer denselben Trainingsdatensatz, sondern entnimmt **wiederholte Bootstrap-Stichproben** aus einem Trainingssatz D .
- **Bootstrap-Stichproben**: Ausgehend von der Menge D , die N Trainingsbeispiele enthält, wird D' erstellt, indem N Beispiele **zufällig mit Zurücklegen** aus D gezogen werden.

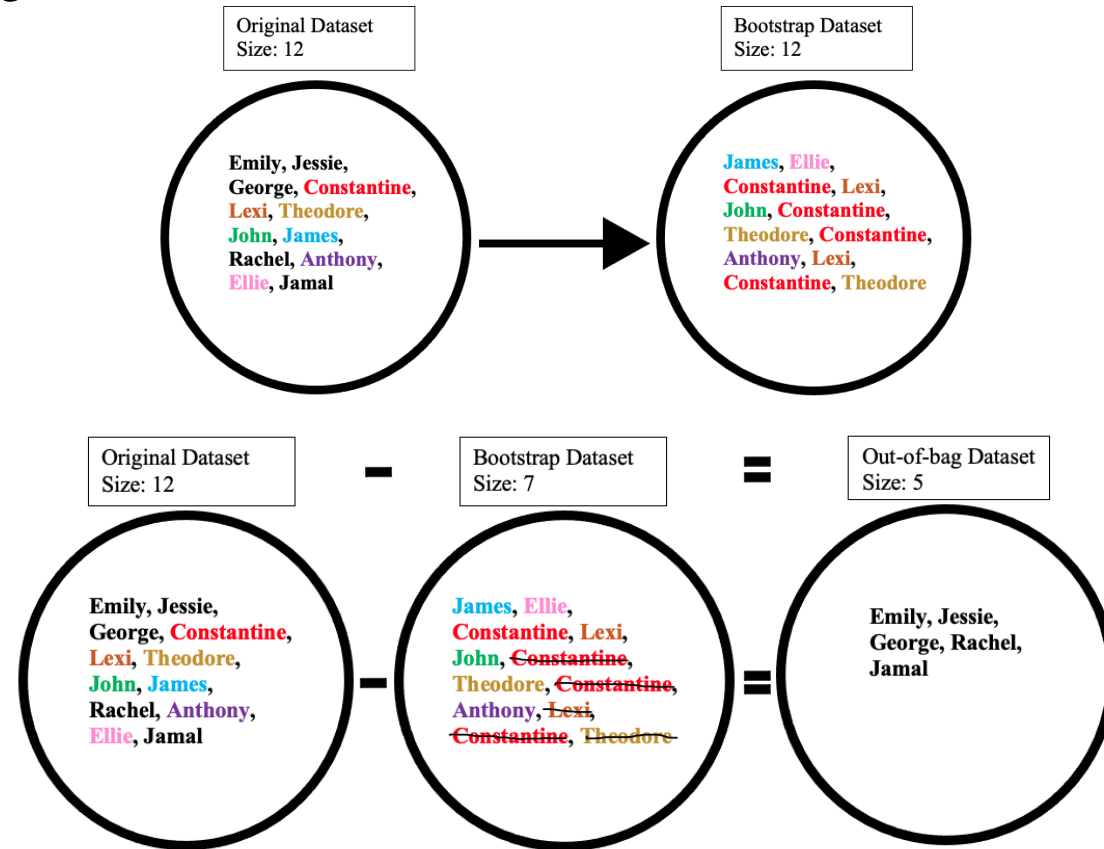
Bagging-Verfahren:

1. Erstelle **k** Bootstrap-Stichproben $D_1 \dots D_k$.
2. Trainiere für jedes D_i **parallel** einen eigenen Klassifikator
3. Klassifiziere neue Instanzen nach **Mehrheitsbeschluss bzw. Durchschnitt**.



Bagging

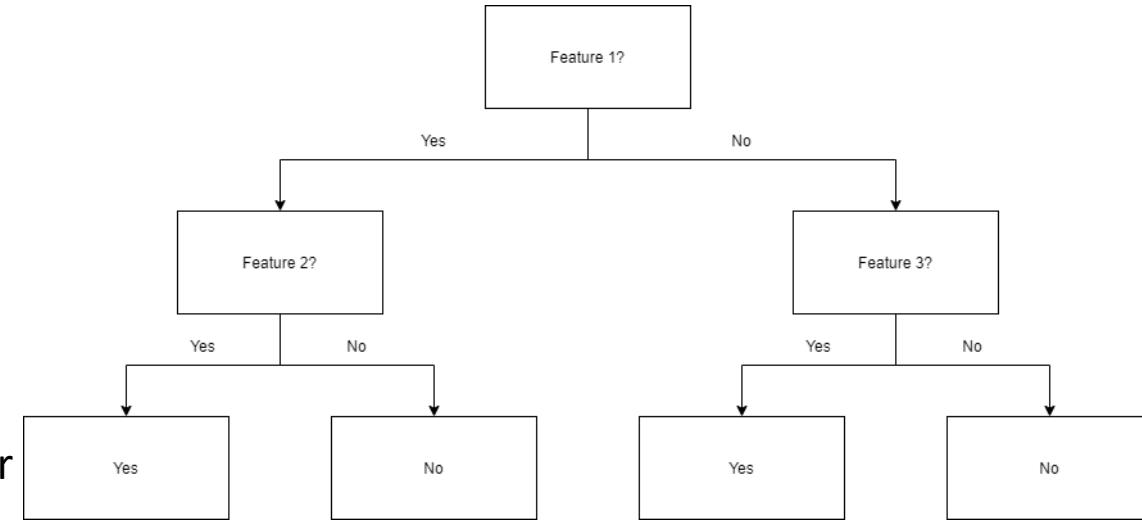
- **Bootstrap-Stichprobenverfahren:** Ausgehend von der Menge D , die N Trainingsbeispiele enthält, wird D' gebildet, indem N Beispiele zufällig mit Zurücklegen aus D gezogen werden.
- Die Größe des Bootstrap-Datensatzes muss mit der des ursprünglichen Datensatzes übereinstimmen.
- Der Unterschied besteht darin, dass der Bootstrap-Datensatz doppelte Objekte enthalten kann.
- Einige Dateninstanzen aus dem ursprünglichen Datensatz werden nicht verwendet; diese Instanzen sind Teil des sogenannten **Out-of-Bag-Datensatzes**, der zum **Testen der Genauigkeit jedes schwachen Lerner** verwendet wird



Quelle: [Wikipedia](https://de.wikipedia.org/wiki/Bagging)

Bagging

- Beim Bagging sind die Basis-/schwachen Lerner in der Regel **Entscheidungsbäume**.
- Für jeden Trainingssatz (Bootstrap-Stichprobe) und alle Merkmale des ursprünglichen Trainingsdatensatzes erstellt Bagging einen Entscheidungsbaum.
- Nachdem wir **k** Entscheidungsbäume trainiert haben, können wir diese nutzen, um neue Daten nach **der Mehrheitsregel** zu klassifizieren.
- Das heißt, jeder Entscheidungsbaum trifft eine Entscheidung, und wir sagen die Klasse voraus, die mehr Stimmen erhalten hat.



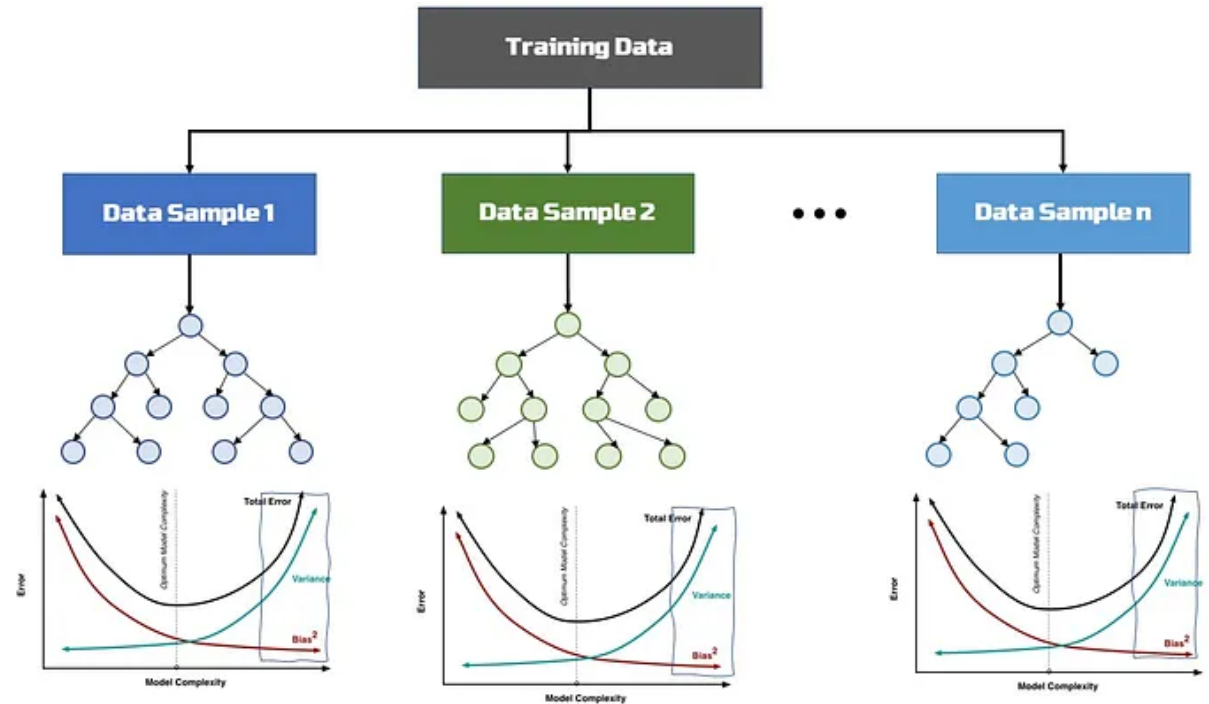
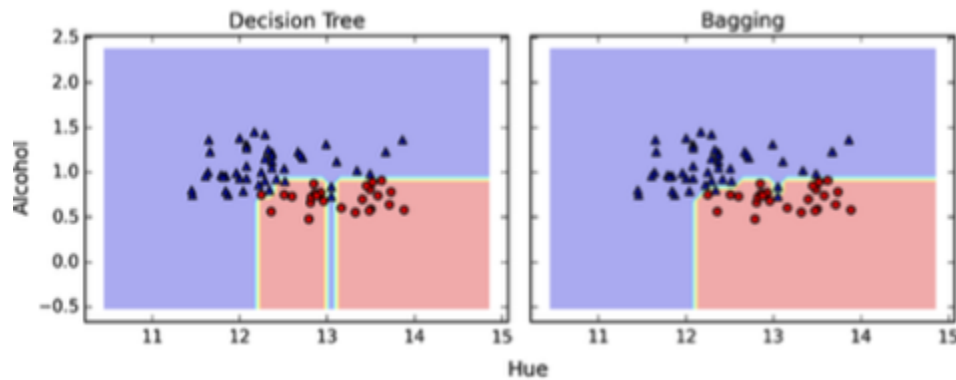
Quelle: [Wikipedia](#)

Unterschied zum Voting:

- Die Trainingsdaten sind nicht für alle Modelle gleich (Bootstrap-Stichproben)
- Alle Basislerner verwenden denselben Algorithmus (Entscheidungsbaum)

Vorteile Bagging

- Bessere Vorhersageleistung
- Verringert das Overfitting → geringere Varianz bei den Vorhersagen
- Weniger komplexe Entscheidungsgrenze



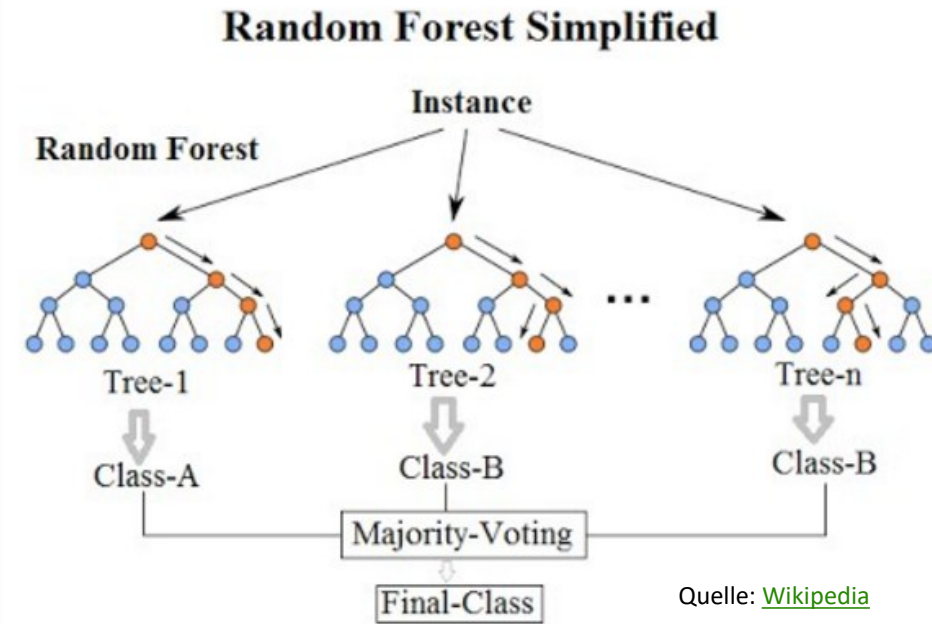
Quelle: <https://sebastianraschka.com/faq/docs/bagging-boosting-rf.html>

Random Forest

- Random Forest ist eine **bekannte** Ensemble-Lernmethode für **Klassifikation** und **Regression**
- Es handelt sich um eine Erweiterung von Bagging

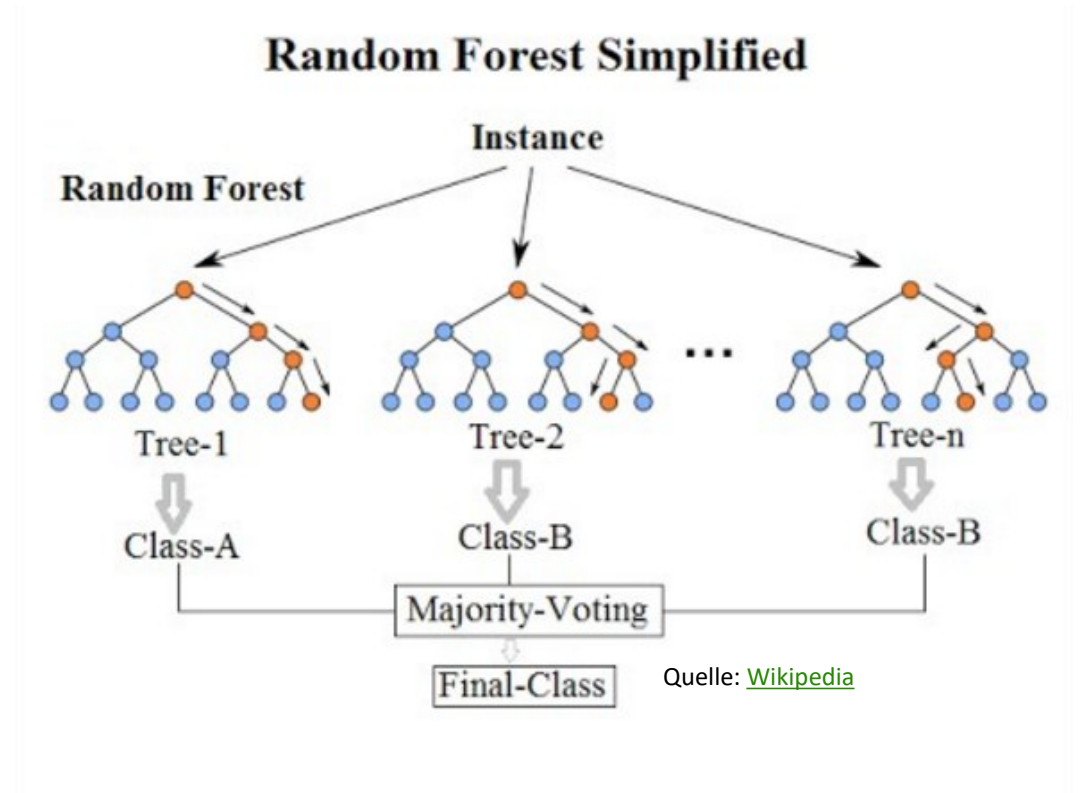
Schritte des Algorithmus:

1. Angenommen, der Trainingsdatensatz enthält N Beobachtungen und M Merkmale. Zunächst werden **k Stichproben der Größe N zufällig** aus dem Trainingsdatensatz entnommen (mit Zurücklegen)
2. Für jede Teilstichprobe trainieren wir einen Entscheidungsbaum:
 - An jedem Knoten wählen wir eine **Teilmenge der M Merkmale** aus (es werden nicht alle Merkmale verwendet)
 - **Das Merkmal, das die beste Teilung erzeugt, wird ausgewählt**
 - Der Baum wird erweitert.
3. Die obigen Schritte werden wiederholt, bis die gewählte Anzahl an Bäumen erreicht ist
4. Die Vorhersage wird durch Mehrheitsentscheid oder Durchschnitt der Vorhersagen aller Bäume berechnet.



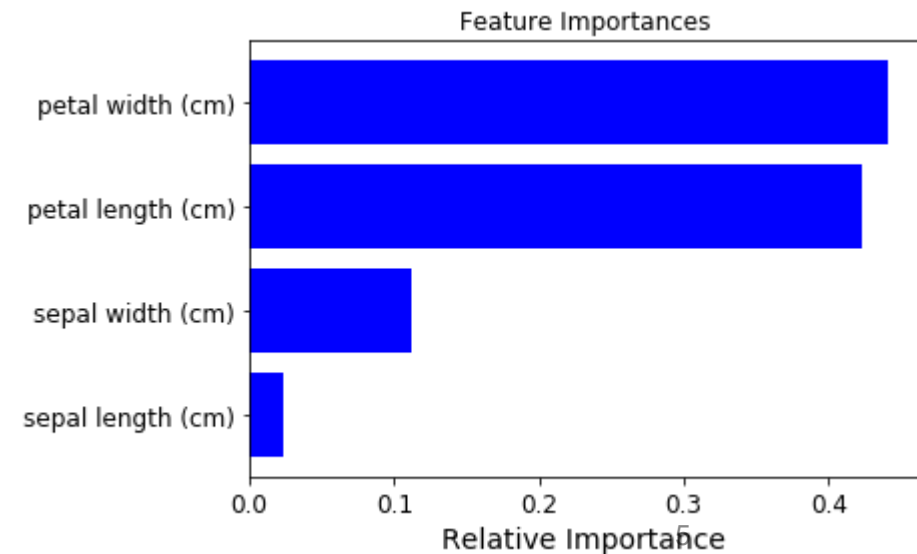
Random Forest vs. Bagging

- Zusätzlich zu den Bootstrap-Stichproben ziehen wir auch **zufällige Teilmengen von Merkmalen für das Training** der einzelnen Bäume (beim Bagging stellen wir jedem Baum den vollständigen Satz an Merkmalen zur Verfügung).
- Aufgrund der **zufälligen Merkmalsauswahl** sind die Bäume im Vergleich zum regulären Bagging **unabhängiger voneinander**, was oft zu **einer besseren Vorhersageleistung** führt (aufgrund des besseren Varianz-Bias-Tradeoffs).
- Es ist **schneller** als Bagging, da jeder Baum nur aus einer Teilmenge von Merkmalen gelernt wird.



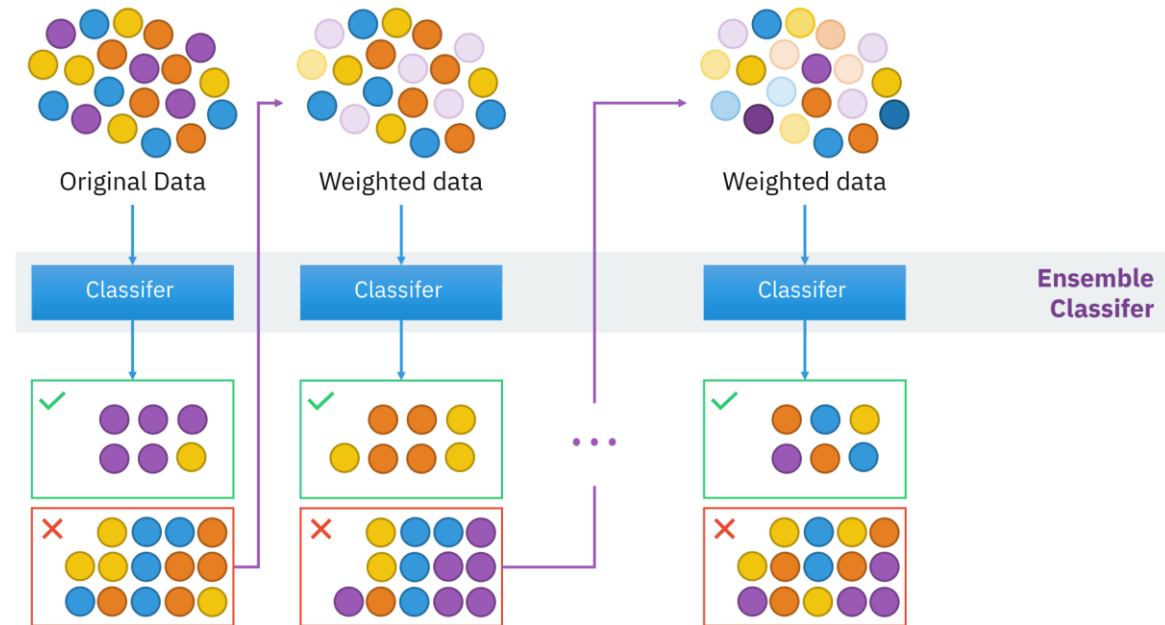
Random Forest – Vorteile und Nachteile

- + Oft **sehr hohe Vorhersagekraft** (hohe Genauigkeit) aufgrund der Reduzierung des Varianzfehlers
- + Kann für Regressions- und Klassifizierungsaufgaben verwendet werden
- + Random Forest kann fehlende Werte in den Daten automatisch verarbeiten und ist robust gegenüber Ausreißern
- + Keine Skalierung der Merkmale erforderlich
- + Berechnet **die feature importance**, wodurch Random Forest besser **erklärbar** ist als andere Ensemble-Lernalgorithmen (wie Voting)
- Hohe Komplexität – es werden viele Bäume erstellt
- Längere Trainingsdauer



Boosting

- **Boosting** ist ein **sequenzieller** Ensemble Algorithmus, der in erster Linie darauf abzielt, **den Bias zu reduzieren**
- Erfunden von Robert Schapire im Jahr 1990
- Boosting-Algorithmen zielen darauf ab, die Vorhersagekraft zu verbessern, indem **sie eine Abfolge schwacher Modelle trainieren, von denen jedes die Schwächen seiner Vorgänger ausgleicht** (sie arbeiten nicht parallel wie beim Bagging)
- Trainingsinstanzen mit hohen Fehlern werden **stärker gewichtet** als solche mit geringeren Fehlern (bei AdaBoost)
- Es gibt viele verschiedene Varianten, z. B.:
 - ADABOOST
 - LogitBoost
 - BrownBoost
 - GradientBoost – XGBoost
 - ...



Quelle: [Wikipedia](#)

Boosting

- **ADA Boost = Adaptive Boosting** ist eine der ältesten klassischen Varianten des Boosting
- Anwendbar auf Klassifikation

Schritte von ADA Boost:

1. Weise den Trainingsbeispielen **gleiche anfängliche Datengewichte** zu. Die Gewichte jedes Beispiels geben an, wie wichtig es ist, dass es korrekt klassifiziert wird.
2. **Ein schwacher Lerner F_s wird trainiert** (z. B. Decision Stump – ein Entscheidungsbaum mit nur einer Variable)
3. Wir bewerten die Fehler err_s für jedes Decision-Stump-Modell.
4. Wir berechnen die sogenannten „Konfidenz“-Koeffizienten w_s für jeden Klassifikator auf der Grundlage der Fehler. Niedrige Fehlerraten führen zu einem hohen Koeffizienten, was **eine höheres Gewicht** in der Kombination der Klassifikatoren bedeutet.

$$w_s = \frac{1}{2} \log \left(\frac{1 - err_s}{err_s} \right)$$

Boosting

5. **Die Datengewichte werden** auf der Grundlage der gemachten Fehler **aktualisiert**. Den falsch klassifizierten Stichproben wird mehr Gewicht zugewiesen, damit sie im nächsten Entscheidungstumpf korrekt klassifiziert werden.
6. Wiederhole ab Schritt 2, bis alle Datenpunkte korrekt klassifiziert wurden oder die **maximale Anzahl von Iterationen** s erreicht wurde.
7. **Berechne Vorhersagen** durch **gewichtete (unter Verwendung von w_s) Mehrheitsentscheidung** aller Modelle

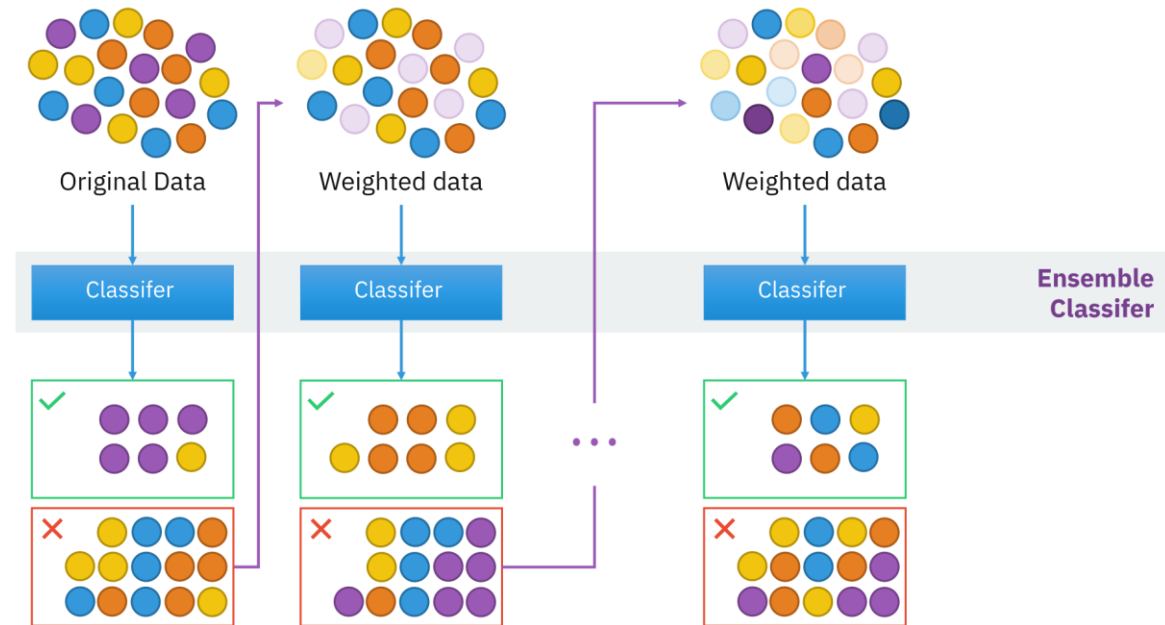
$$F_s(x) := \sum_{i=1}^s w_i f_i(x)$$

XGBoost

- XGBoost – eXtreme Gradient Boosting ist für Klassifikation und Regression geeignet
- Im Vergleich zu AdaBoost gewichtet XGBoost nicht einzelne Datenpunkte sondern lernt aus den Gradienten der Fehlerfunktion = minimiert die Residuen (Fehler) durch Gradientenabstieg
- XGBoost nutzt Regularisierung (beschränkt Baumtiefe und verhindert Overfitting)
- Nutzt nicht nur Decision Stumps sondern tiefe Bäume
- Ist sehr effizient und skalierbar, aber speicherintensiv
- Sehr hohe Vorhersagegenauigkeit für tabellarische Daten (viele Challenges wurden mit XGBoost gewonnen)
- Hat viele Hyperparameter

Boosting Vor- und Nachteile

- + Sehr gute Vorhersageleistung
- + Reduziert Bias
- Reagiert empfindlich auf Ausreißer
- Hoher Rechenaufwand



Quelle: [Wikipedia](#)

Vergleich Ensemble Learning Verfahren

Ensemble-Methode	Verwendete Daten	Basis-Lernmodell	Lernablauf	Fehlerreduktion
Voting	Alle	Verschiedene Algorithmen	parallel	Varianz
Bagging	Zufallsauswahl	Gleicher Algorithmus	parallel	Varianz
Boosting	Gewichtete Daten	Gleicher Algorithmus	Sequenziell	Bias (& Varianz)

Zusammenfassung

- Ensemble-Verfahren lernen mehrere Modelle und kombinieren deren Vorhersageergebnisse.
- Das Ziel besteht darin, die Vorhersageleistung durch Verringerung von Bias und Varianz (Bias-Varianz-Tradeoff) zu verbessern.

- + Ensemble-Lernen **erhöht** häufig die **Vorhersageleistung**
- + Viele Ensemble-Lernalgorithmen **reduzieren den Varianzfehler**, nur Boosting reduziert den **Bias-Fehler**
- Da wir mehrere Modelle trainieren, erhöht sich die **Rechenzeit**
- Weniger interpretierbar als manche Einzelmodelle

